Chapter 6 — Strings and Regular Expressions

Text is everywhere: user names, URLs (web addresses), log lines, CSV files (comma-separated values — spreadsheet-like plain text), chat messages, HTML. This chapter covers **strings** (how JavaScript represents text), the everyday methods to search, cut, change and compare it, the Unicode details that trip up experienced developers (why `'😀'.length` is 2), and **regular expressions** — a compact pattern language for finding and transforming text.

What you'll learn: quotes, escapes, template literals (interpolation, multi-line, tags, `String.raw`); indexing, `length`, `at()`, immutability, coercion; primitives vs `String` objects; the methods table; Unicode (code units, code points, graphemes, normalization, `Intl.Segmenter`, well-formed strings); regex syntax, flags, lookaround; regex methods and the `lastIndex` trap; modern features (named groups, `d` / `u` / `v` flags, `RegExp.escape`, modifiers, duplicate named groups).

Before you start: variables and types (Chapter 2), functions (Chapter 4); array methods like `map` / `join` are in Chapter 7.

Code is in `code/`; files are ES modules (always strict mode — Chapter 1); outputs are real Node 24 output.

6.1 What a string is

What it is

A **string** is an ordered, **immutable** sequence of characters — a **primitive** (`typeof 'hi'` is `'string'`). No operation changes a string in place; every method returns a *new* string. Precisely, it is a sequence of **UTF-16 code units** (16-bit numbers); for everyday English one unit = one character (§6.5 covers when that breaks).

Why it matters

Most input arrives as text — form fields, query strings (the `?key=value&…` part of a URL), JSON, files — and must be cleaned, validated, split and reformatted. Trailing spaces, case differences and emoji breaking length limits are among the most common web-app bugs.

6.2 Creating strings

Three kinds of quotes

Delimiter	Example	Interpolation `${}`	Real line breaks allowed	Typical use
Single quotes `'…'`	`'It works'`	No	No	Plain strings (common default in style guides — shared formatting rulebooks — e.g. Airbnb/StandardJS)
Double quotes `"…"`	`"It works"`	No	No	Plain strings (Prettier's default); identical behaviour to single quotes
Backticks `` `…` ``	`` `Hi ${name}` ``	**Yes**	**Yes**	Anything built from values, multi-line text, tagged templates

Single and double quotes are equivalent; a formatter (Prettier — Chapter 10) enforces one.

Quotes inside strings

Use a *different* outer quote, or **escape** the inner quote with a backslash. From `code/01_string_basics.mjs`:

```
console.log("She said 'hi'", '|', 'He said "bye"', '|', 'It\'s escaped', '|', `mix 'both' "kinds"`);
```

Output:

```
She said 'hi' | He said "bye" | It's escaped | mix 'both' "kinds"
```

Escape sequences

A backslash starts an **escape sequence** — source characters standing for one special character.

Escape	Produces	Notes
`\'` `\"` `\``	The quote character itself	Needed only when it matches the delimiter
`\\`	One backslash	Windows paths: `'C:\\temp'`
`\n`	Line feed (new line)	The normal line break on Linux/macOS and in most text protocols (message formats like HTTP)
`\r`	Carriage return	Windows line endings are `\r\n`
`\t`	Horizontal tab	
`\b`	Backspace (U+0008)	Rare. Careful: inside a **regex** (§6.7), `\b` means "word boundary"
`\f`	Form feed (U+000C)	Rare (old printers)
`\v`	Vertical tab (U+000B)	Rare; supported by all modern engines
`\0`	Null character (U+0000)	Must not be followed by a digit (`'\01'` is a legacy octal — base-8 — escape, a `SyntaxError` in strict mode)
`\xHH`	Character with hex (base-16) code HH (2 digits)	`'\x41'` is `'A'`
`\uXXXX`	UTF-16 code unit with hex code XXXX (4 digits)	`'\u00e9'` is `'é'`
`\u{X…}`	Any Unicode code point (§6.5; 1–6 hex digits), ES2015	`'\u{1F600}'` is `'😀'`
`\` + line break	Line continuation (no character added)	Old way to split a long literal across lines

```
console.log('Tab:\t|  Backslash: \\  Unicode: \u00e9 \u{1F600}  Hex: \x41');
console.log('Line 1\nLine 2');
console.log(JSON.stringify('a\r\nb\0c'), 'a\r\nb'.length);
```

Output:

```
Tab:    |  Backslash: \  Unicode: é 😀  Hex: A
Line 1
Line 2
"a\r\nb\u0000c" 4
```

Each escape is one character in the value: `'a\r\nb'.length` is 4.

6.3 Template literals

What it is

A **template literal** (ES2015, backticks) adds:

1. **Interpolation** — `${expression}` inserts any expression's value, converted to string.
2. **Multi-line text** — real line breaks are kept.
3. **Tags** — a function before the backtick receives the pieces and values and builds any result.

How it works

```
const item = 'coffee', price = 3.5, qty = 2;
console.log(`${qty} x ${item} = $${(price * qty).toFixed(2)} ${qty > 1 ? '(bulk)' : ''}`);
const multi = `Dear user,
  your order has shipped.`;
console.log(multi);
```

Output:

```
2 x coffee = $7.00 (bulk)
Dear user,
  your order has shipped.
```

- In `$${…}` the first `$` is literal; `${` starts interpolation.
- Any expression works inside `${}` — arithmetic, calls, ternaries, nested templates.
- **Indentation inside a multi-line template is part of the string.**

Templates replaced the pre-ES2015 `'line 1\n' + 'line 2'` style.

Tagged templates and `String.raw`

A **tag** is called with (a) an array of literal text pieces and (b) the interpolated values. Libraries use tags to escape HTML, build SQL (database query language) safely, or define CSS: styled-components' `styled.div`…``, GraphQL's (an API query language) `gql`…``, lit-html's `html`…``.

```
function highlight(strings, ...values) {
  return strings.reduce((out, s, i) => out + s + (i < values.length ? `[${values[i]}]` : ''), '');
}
console.log(highlight`Hello ${'Ana'}, you have ${3} messages`);
console.log(String.raw`C:\new\table`, '|', 'C:\new\table'.length, String.raw`C:\new\table`.length);
```

Output:

```
Hello [Ana], you have [3] messages
C:\new\table | 10 12
```

`strings` is `['Hello ', ', you have ', ' messages']`, `values` `['Ana', 3]` — always one more piece than values. `String.raw` returns text *without* processing escapes (handy for Windows paths and regex sources); the normal `'C:\new\table'` has 10 characters because `\n` and `\t` became newline and tab.

Common mistakes

- `'${name}'` in normal quotes — only backticks interpolate.
- HTML from user input via templates without escaping → **XSS** (cross-site scripting). Escape or use `textContent` (Chapter 14).
- SQL by interpolation → **SQL injection** (attacker input runs as part of your command). Use parameterised queries (command and values sent separately, so values are never code).

6.4 Working with strings: indexing, immutability, coercion, String objects

Indexing, `length` and `at()`

Characters are numbered from **0**: `str[i]`, `str.charAt(i)`, or `str.at(i)` (ES2022), which accepts **negative** indexes from the end.

```
const greeting = 'Hello, World!';
console.log(greeting.length, greeting[0], greeting[7], greeting.at(-1), greeting[50], greeting.charAt(50) === '');
```

Output:

```
13 H W ! undefined true
```

`length` counts code units (spaces and punctuation included); last index is `length - 1` (or `at(-1)`); past the end, brackets give `undefined`, `charAt` gives `''`.

Strings are immutable

```
const s = 'cat';
try { s[0] = 'b'; } catch (e) { console.log('assigning s[0] ->', e.constructor.name); }
console.log(s, 'b' + s.slice(1));
```

Output:

```
assigning s[0] -> TypeError
cat bat
```

Index assignment is silently ignored in **sloppy** scripts and throws `TypeError` in **strict** mode. Build a new string instead.

Concatenation and coercion

`+` concatenates once either side is a string (`'5' + 3` → `'53'`; left to right, so `1 + 2 + '3'` is `3 + '3'` → `'33'` but `'1' + 2 + 3` is `'12' + 3` → `'123'`); `-`, `*`, `/` convert to numbers (`'5' - 3` → `2`) — full rules in Chapter 2 §2.5. Convert **to** string with `String(x)` (safe for `null`/`undefined`; `String([1, 2])` → `'1,2'`, `String({})` → `'[object Object]'`) or a template; `x.toString()` throws on `null`/`undefined`.

String primitives vs `String` objects

`new String('…')` creates a **wrapper object** — essentially never wanted.

```
const obj = new String('Hello');
const prim = 'Hello';
console.log(typeof obj, typeof prim, obj === prim, obj == prim, obj.valueOf() === prim);
console.log(new String('a') === new String('a'), typeof String(123));
console.log(prim.toUpperCase(), 'methods work on primitives via temporary wrapper objects');
```

Output:

```
object string false true true
false string
HELLO methods work on primitives via temporary wrapper objects
```

	String primitive `'Hello'`	**String object `new String('Hello')`**
`typeof`	`'string'`	`'object'`
Equality	By value — two equal primitives are `===`	By reference — two wrappers are never `===`, even with the same text
Truthiness	`''` is falsy	Every object is truthy — `new String('')` is truthy!
Cost	Cheap; engines optimise heavily	Extra allocation (memory for a new object)
Methods	Available through **auto-boxing**	Available directly

Auto-boxing: for `'Hello'.toUpperCase()` the engine briefly wraps the primitive to find the method on `String.prototype` (where all strings' methods live — Chapter 9), then discards the wrapper. `String(123)` **without** `new` converts and returns a primitive.

Comparing strings

```
console.log('apple' < 'banana', 'Zebra' < 'apple', 'a'.localeCompare('B'), ['é', 'z', 'e'].sort(), ['é', 'z', 'e'].sort((a, b) => a.localeCompare(b)));
```

Output:

```
true true -1 [ 'e', 'z', 'é' ] [ 'e', 'é', 'z' ]
```

`<`, `>` and default `sort()` compare **code unit by code unit**: all uppercase before lowercase (`'Z'` 90, `'a'` 97), accented letters after `z`. For human ordering use **`localeCompare`** or `Intl.Collator` (§6.6; `Intl` is the built-in internationalisation namespace).

Self-check

1. `'abc'[-1]` vs `'abc'.at(-1)`? → `undefined` vs `'c'`.
2. `'3' + 4 + 5`? → `'345'`; `3 + 4 + '5'`? → `'75'`.
3. `new String('') ? 'y' : 'n'`? → `'y'` (objects are truthy).
4. Why is `'B' < 'a'`? → Code unit 66 < 97.

6.5 Unicode: code units, code points and graphemes

The problem it solves

Why is `'😀'.length` 2? Why can reversing a string produce garbage? Why are `'café'` and `'café'` sometimes unequal? All about encoding.

Three levels of "character"

Level	What it is	Example	JavaScript view
Code unit	One 16-bit number in UTF-16	`'😀'` = two units `D83D DE00`	`length`, `str[i]`, `charCodeAt`, `slice` indexes
Code point	One Unicode character number (U+0000 … U+10FFFF)	`'😀'` = U+1F600	`codePointAt`, `String.fromCodePoint`, `for…of`, `[...str]`
Grapheme cluster	What a person sees as one character	`'👨‍👩‍👧'` = 5 code points, 1 grapheme	`Intl.Segmenter`

Code points up to U+FFFF (the **Basic Multilingual Plane**, most living scripts) fit one code unit. Above it — most emoji, rare CJK (Chinese/Japanese/Korean) characters, music and math symbols — use a **surrogate pair**: a "high" unit D800–DBFF then a "low" DC00–DFFF. A lone half is a **lone surrogate**; a string containing one is **not well-formed**.

From `code/03_unicode.mjs`:

```
const accented = 'café';                    // é as one code point U+00E9
const decomposed = 'cafe\u0301';            // e + combining acute accent U+0301
console.log(accented, decomposed, accented === decomposed, accented.length, decomposed.length);
console.log(accented === decomposed.normalize('NFC'), accented.normalize('NFD').length);

const smile = '😀';
console.log(smile.length, smile.charCodeAt(0).toString(16), smile.charCodeAt(1).toString(16));
console.log(smile.codePointAt(0).toString(16), String.fromCodePoint(0x1f600) === smile, '\u{1F600}' === smile);
console.log(JSON.stringify(smile[0]), smile[0].isWellFormed(), smile.isWellFormed());

const word = 'hi😀!';
console.log(word.length, [...word].length, [...word]);
console.log('naive reverse :', word.split('').reverse().join('').isWellFormed());
console.log('safer reverse :', [...word].reverse().join(''));
```

Output:

```
café café false 4 5
true 5
2 d83d de00
1f600 true true
"\ud83d" false true
5 4 [ 'h', 'i', '😀', '!' ]
naive reverse : false
safer reverse : !😀ih
```

1. **Normalization:** "é" can be one code point (U+00E9) or `e` + combining accent (U+0301) — identical-looking, not `===`. `normalize('NFC')` composes, `'NFD'` decomposes; normalize user input (e.g. usernames) before comparing/storing. `'NFKC'`/`'NFKD'` also fold "compatibility" variants (`fi` → `fi`).
2. **Surrogate pairs:** `smile[0]` is a lone surrogate; `codePointAt`/`fromCodePoint`/`\u{…}` handle whole code points.
3. **Iteration by code point:** `for…of`, `[...str]`, `Array.from(str)` split correctly; `split('')` splits code units and can cut pairs, breaking a reversed string.

Grapheme clusters and `Intl.Segmenter`

Some visible characters are several code points: family emoji (joined by the invisible **zero-width joiner** U+200D), flags (two "regional indicator" letters), skin tones, letters plus combining accents. `Intl.Segmenter` (modern browsers and Node) splits text into graphemes, words or sentences by Unicode rules:

```
const family = '👨‍👩‍👧';
const flag = '🇮🇳';
console.log(family.length, [...family].length, flag.length, [...flag].length);
const seg = new Intl.Segmenter('en', { granularity: 'grapheme' });
const graphemes = (s) => [...seg.segment(s)].map((x) => x.segment);
console.log(graphemes(`a${family}b${flag}`).length, graphemes('cafe\u0301').length);

const words = [...new Intl.Segmenter('en', { granularity: 'word' }).segment('Hello, big world!')]
  .filter((x) => x.isWordLike).map((x) => x.segment);
console.log(words);
```

Output:

```
8 5 4 2
4 4
[ 'Hello', 'big', 'world' ]
```

The family emoji: 8 code units, 5 code points, **1** grapheme. Use graphemes for user-facing limits ("max 280 characters"), cursor movement and truncation.

Well-formed strings and byte length

```
const broken = 'ab\uD800';
console.log(broken.isWellFormed(), broken.toWellFormed().codePointAt(2).toString(16));
console.log(encodeURIComponent(broken.toWellFormed()));
try { encodeURIComponent(broken); } catch (e) { console.log('encodeURIComponent(lone surrogate) ->',
e.constructor.name); }

const enc = new TextEncoder();
for (const s of ['abc', 'café', '😀']) console.log(s, 'length', s.length, 'utf8 bytes', enc.encode(s).length);
```

Output:

```
false fffd
ab%EF%BF%BD
encodeURIComponent(lone surrogate) -> URIError
abc length 3 utf8 bytes 3
café length 4 utf8 bytes 5
😀 length 2 utf8 bytes 4
```

- `isWellFormed()` / `toWellFormed()` (ES2024) detect and repair lone surrogates (→ U+FFFD, the replacement character `&#xFFFD;`). Encoding APIs — `encodeURIComponent` (text → URL-safe `%XX` form), `TextEncoder`, `fetch` bodies — throw or silently substitute on broken strings.
- Files, network **payloads** (data in a message) and DB columns are measured in **UTF-8 bytes** (a **byte** is 8 bits; **UTF-8**, the web's common encoding, uses 1–4 bytes per character), not `length`. Use `new TextEncoder().encode(s).length` (browser) or `Buffer.byteLength(s)` (Node).

In the interview

30-second answer: "JS strings are UTF-16 code units; `length` and indexing count units, so characters beyond U+FFFF like most emoji count as 2. `for…of` and spread iterate code points; `Intl.Segmenter` gives user-perceived characters. Normalize with `normalize('NFC')` before comparing."

- *Reverse safely?* `[...s].reverse().join('')` for code points; reverse `Intl.Segmenter` graphemes for combining marks and ZWJ emoji.
- *Count characters for a tweet-like limit?* Graphemes, not `length`.

Self-check

1. `'😀'.length`? → 2. 2. `[...'😀'].length`? → 1.
2. Identical-looking strings unequal? → Different normalization forms.
3. Word list in any language? → `Intl.Segmenter` with `granularity: 'word'`.

6.6 String methods

How to read this section

Every method **returns a new value**. Index arguments are code-unit positions; an `end` index is **exclusive**.

Job	Method	Returns	Notes
Size	`length` (property)	number of code units	not a method — no `()`
Read a char	`charAt(i)`	1-char string, or `''` if out of range	
	`at(i)` (ES2022)	1-char string or `undefined`; negative `i` counts from end	
	`charCodeAt(i)`	UTF-16 code unit number 0–65535	`codePointAt(i)` for full code points
	`String.fromCharCode(...codes)` / `String.fromCodePoint(...cps)`	new string	**static**: called on `String`, not on a string
Search	`indexOf(sub, from?)` / `lastIndexOf(sub, from?)`	index or **-1**	case-sensitive
	`includes(sub, from?)`	boolean	ES2015
	`startsWith(sub, pos?)` / `endsWith(sub, len?)`	boolean	ES2015
	`search(regex)`	index of first regex match or -1	
	`match(regex)` / `matchAll(regex)`	match info / iterator (hands out results one at a time, Chapter 8)	§6.8
Extract	`slice(start, end?)`	substring; **negative** indexes count from end	the one to use
	`substring(start, end?)`	substring; negatives become 0; swaps args if start > end	older
	`substr(start, length?)`	substring by **length**	legacy (Annex B, the spec's appendix of old web-compatibility features, "normative optional") — avoid
Case	`toUpperCase()` / `toLowerCase()`	new string	locale-independent Unicode rules
	`toLocaleUpperCase(loc?)` / `toLocaleLowerCase(loc?)`	new string	locale-specific rules (a locale is a language + region such as `tr-TR`; e.g. Turkish dotted İ)
Whitespace	`trim()` / `trimStart()` / `trimEnd()`	new string	removes spaces, tabs, newlines and other Unicode whitespace
Padding	`padStart(len, fill?)` / `padEnd(len, fill?)`	new string of at least `len`	ES2017
Build	`repeat(n)`	`n` copies joined	
	`concat(...parts)`	joined string	`+` or templates are more common
Split / join	`split(sep?, limit?)`	**array** of pieces	`sep` can be a string or regex; `arr.join(sep)` reverses it
Replace	`replace(pattern, replacement)`	new string	string pattern → first occurrence only
	`replaceAll(pattern, replacement)` (ES2021)	new string	every occurrence; a regex must have `g`

Job	Method	Returns	Notes
Compare	`localeCompare(other, locale?, options?)`	negative / 0 / positive	language-aware ordering
Unicode	`normalize(form?)`, `isWellFormed()`, `toWellFormed()`	string / boolean	§6.5
Convert	`toString()` / `valueOf()`	the primitive string	mainly for `String` objects

Searching

From `code/02_string_methods.mjs`:

```
const log = 'ERROR 42: disk full; ERROR 7: fan';
console.log(log.indexOf('ERROR'), log.lastIndexOf('ERROR'), log.indexOf('WARN'), log.indexOf('ERROR', 1));
console.log(log.includes('disk'), log.startsWith('ERROR'), log.endsWith('fan'), log.startsWith('42', 6));
console.log(log.search(/\d+/));
```

Output:

```
0 21 -1 21
true true true true
6
```

Use `includes` for yes/no (`indexOf(...) !== -1` is the older idiom). Bug: `if (s.indexOf(x))` is falsy when found **at index 0** and truthy (−1) when missing.

Extracting: `slice` vs `substring` vs `substr`

```
const text = 'Hello, World!';
console.log(text.slice(0, 5), '|', text.slice(7), '|', text.slice(-6, -1));
console.log(text.substring(0, 5), '|', text.substring(5, 0), '|', text.substring(-3, 2));
console.log(text.slice(5, 0) === '', text.substr(7, 5), '(substr: legacy, avoid)');
console.log(text.charAt(4), text.at(-1), text.charCodeAt(0), String.fromCharCode(72, 105));
```

Output:

```
Hello | World! | World
Hello | Hello | He
true World (substr: legacy, avoid)
o ! 72 Hi
```

- `slice(0, 5)` → indexes 0–4 (end excluded); omit `end` to go to the end.
- `slice` accepts negatives; `substring` turns them into 0 and swaps reversed arguments; `slice` with start > end gives `''`.
- `substr(7, 5)` = "5 characters from 7" — legacy; use `slice`.

Case, whitespace and padding

```
console.log(text.toUpperCase(), text.toLowerCase());
console.log('istanbul'.toLocaleUpperCase('tr-TR'), 'istanbul'.toUpperCase());
console.log('Straße'.toUpperCase(), 'ß'.toLowerCase());

const raw = '   padded text \n';
console.log(JSON.stringify(raw.trim()), JSON.stringify(raw.trimStart()), JSON.stringify(raw.trimEnd()));
console.log('7'.padStart(3, '0'), 'ab'.padEnd(5, '.') + '|', '4111222233334444'.slice(-4).padStart(16, '*'));
```

Output:

```
HELLO, WORLD! hello, world!
İSTANBUL ISTANBUL
STRASSE ȸ
"padded text" "padded text \n" "   padded text"
007 ab...| ************4444
```

- Case conversion is Unicode-aware and can change length (`ß` → `SS`); only the locale-aware version produces Turkish `İ`.
- **`Case-insensitive comparison`**: lowercase both for simple text; for accents/other languages `a.localeCompare(b, undefined, { sensitivity: 'base' }) === 0`.
- `padStart` for zero-padding and masking card numbers.

Building, splitting and replacing

```
console.log('ab'.repeat(3), 'a'.concat('b', 'c', 1), ['x', 'y', 'z'].join('-'));
console.log('a,b,,c'.split(','), 'hello'.split(''), 'a b  c'.split(' '), 'a b  c'.split(/\s+/), 'x'.split(','),
'a,b,c'.split(',', 2));
const s = 'cat, cat, cat';
console.log(s.replace('cat', 'dog'), '|', s.replaceAll('cat', 'dog'), '|', s.replace(/cat/g, 'cow'));
```

Output:

```
ababab abc1 x-y-z
[ 'a', 'b', '', 'c' ] [ 'h', 'e', 'l', 'l', 'o' ] [ 'a', 'b', '', 'c' ] [ 'a', 'b', 'c' ] [ 'x' ] [ 'a', 'b' ]
dog, cat, cat | dog, dog, dog | cow, cow, cow
```

- `split` keeps empty strings between consecutive separators; a regex like `/\s+/` collapses runs. No match → one-element array; the second argument limits pieces.
- **`replace` with a string pattern changes only the first occurrence** — use `replaceAll` or a `/g` regex.

Comparing with `localeCompare` and `Intl.Collator`

```
console.log('a'.localeCompare('b'), 'b'.localeCompare('a'), 'a'.localeCompare('a'));
console.log('résumé'.localeCompare('resume', 'en', { sensitivity: 'base' }), 'ä'.localeCompare('z', 'de'),
'ä'.localeCompare('z', 'sv'));
const collator = new Intl.Collator('en', { numeric: true });
console.log(['file10', 'file2', 'file1'].sort(), ['file10', 'file2', 'file1'].sort(collator.compare));
```

Output:

```
-1 1 0
0 -1 1
[ 'file1', 'file10', 'file2' ] [ 'file1', 'file2', 'file10' ]
```

`localeCompare` returns negative/0/positive — exactly what `sort` expects from a **`comparator`** (function deciding which item comes first). Order depends on **`language`**: German sorts `ä` with `a`, Swedish after `z`. `sensitivity: 'base'` ignores accents and case; `numeric: true` gives **`natural sort`** (`file2` before `file10`). For many strings reuse one `Intl.Collator`'s `compare` — much faster than repeated `localeCompare`.

Converting strings to numbers

`Number(str)` is strict (`Number('42px')` → `NaN`; whole trimmed string must be numeric; `''` → `0`); `parseInt('42px', 10)` → `42` and `parseFloat('3.14abc')` → `3.14` read leading digits and stop. Always pass the **`radix`** (number base) to `parseInt`. Details: Chapter 2 §2.10. (`toString()`/`valueOf()` on a `String` wrapper return the primitive.)

Two real tasks

```
const slugify = (title) => title.trim().toLowerCase()
  .normalize('NFKD').replace(/[\u0300-\u036f]/g, '')   // strip accents
  .replace(/[^a-z0-9]+/g, '-').replace(/^-+|-+$/g, '');
console.log(slugify('  Crème Brûlée: 10 Tips & Tricks!  '));

const titleCase = (str) => str.split(' ').map((w) => w.charAt(0).toUpperCase() + w.slice(1).toLowerCase()).join('
');
console.log(titleCase('the qUICK brown fox'));
```

Output:

```
creme-brulee-10-tips-tricks
The Quick Brown Fox
```

`slugify` builds URL **slugs** (readable address endings like `my-first-post`) as blogs and CMSs (content management systems like WordPress) do: trim, lowercase, decompose accents and delete combining marks (U+0300–U+036F), turn non-alphanumeric runs into `-`, strip edge dashes.

Performance note

`s += piece` in a loop conceptually creates a new string each time, but engines optimise it with internal "rope" structures (trees of pieces joined only when needed). For very large output, pushing pieces to an array and `join('')` once is a predictable alternative.

Common mistakes

- `replace('a', 'b')` expecting every `a` to change.
- `if (str.indexOf(x))` instead of `includes`.
- Comparing user text without `trim()` and case/accent normalization.
- `substr` in new code; confusing `substring(start, end)` with `substr(start, length)`.
- Sorting names with default `sort()`.

In the interview

30-second answer: "Strings are immutable primitives; methods return new strings. `slice` for extraction (negatives OK), `includes` / `startsWith` for tests, `split` / `join` to convert to arrays, `replaceAll` or a `/g` regex to replace everywhere, `padStart` for formatting, `localeCompare` / `Intl.Collator` for human sorting."

- *Palindrome check* (reads the same backwards, like "level"): `const t = s.toLowerCase().replace(/[^a-z0-9]/g, ''); t === [...t].reverse().join('')`.
- *Count occurrences:* `s.split(sub).length - 1`, or `s.match(/sub/g)?.length ?? 0` (escape input patterns — `RegExp.escape`, §6.9).
- *`slice` vs `substring`?* Negative indexes and argument swapping.

Self-check

1. `'JavaScript'.slice(-6)`? → `'Script'`.
2. `'a-b-c'.split('-', 2)`? → `['a', 'b']`.
3. `'5'.padStart(3, '0')`? → `'005'`.
4. Case-insensitive sort of `['b', 'A', 'a']`? → `.sort((x, y) => x.localeCompare(y, undefined, { sensitivity: 'base' }))`.

6.7 Regular expressions: the pattern language

What it is

A **regular expression** (regex, regexp) is a pattern describing a set of strings ("five digits", "anything between `<` and `>`"); in JS a `RegExp` object, run by the regex engine against text from left to right until the pattern fits. Two forms:

- **Literal** `/pattern/flags` — compiled (into the engine's internal matcher) once at load; for fixed patterns.
- **Constructor** `new RegExp('pattern', 'flags')` — built at run time from data; double the backslashes: `new RegExp('\\d+')`.

Why it matters

Form validation, log extraction, search-and-replace, URL routing (`/users/:id` becomes a regex inside **routers**, which pick the handler for each address), syntax highlighting, **tokenizers** (chop text into words, numbers, symbols) — all use regexes, which are easy to get wrong.

Syntax cheat sheet

Category	Syntax	Matches
Literal characters	`abc`	the text `abc`
Any character	`.`	any char except line terminators (unless the `s` flag)
Escaping	`\.` `\*` `\\` `\/`	the special character itself
Character class	`[abc]`, `[a-z0-9]`	one char from the set / range
Negated class	`[^abc]`	one char **not** in the set
Shorthand classes	`\d` `\w` `\s`	digit `[0-9]`; word char `[A-Za-z0-9_]`; whitespace
Negated shorthands	`\D` `\W` `\S`	the complements
Unicode property	`\p{L}`, `\p{Nd}`, `\p{Script=Greek}` (needs `u` or `v`)	any letter / decimal digit / Greek char
Quantifiers	`*` `+` `?`	0 or more / 1 or more / 0 or 1
Counted	`{n}` `{n,}` `{n,m}`	exactly n / at least n / n to m
Lazy quantifiers	`*?` `+?` `??` `{n,m}?`	as **few** as possible
Anchors	`^` `$`	start / end of input (of each line with `m`)
Word boundary	`\b` `\B`	position between `\w` and `\W` / not a boundary
Alternation	`a	b`	
Capturing group	`( … )`	groups and remembers the match as `$1`, `m[1]`, …
Named group	`(?<name> … )`	remembered as `groups.name` / `$<name>`
Non-capturing group	`(?: … )`	groups without remembering
Backreference	`\1`, `\k<name>`	the same text a group matched earlier
Lookahead	`(?= … )` / `(?! … )`	position followed / not followed by …
Lookbehind (ES2018)	`(?<= … )` / `(?<! … )`	position preceded / not preceded by …

Flags

Flag	Name	Effect
`g`	global	Find **all** matches (affects `match`, `replace`, `matchAll`; makes `test`/`exec` stateful — they remember something between calls)
`i`	ignoreCase	Case-insensitive
`m`	multiline	`^` and `$` match at each line start/end
`s`	dotAll (ES2018)	`.` also matches newlines
`u`	unicode (ES2015)	Work in code points; enables `\p{…}` and `\u{…}`; stricter syntax
`v`	unicodeSets (ES2024)	Superset of `u`: set operations in classes, string properties; use `u` **or** `v`, not both
`y`	sticky	Match only at exactly `lastIndex` (used by tokenizers)
`d`	hasIndices (ES2022)	Add start/end `indices` for each group

6.8 Regex methods and techniques

Which method to use

Goal	Use	Returns
Does it match?	`regex.test(str)`	`true` / `false`
First match with details	`str.match(regexWithoutG)` or `regex.exec(str)`	match array (`[0]` full match, `[1]…` groups, `.index`, `.groups`) or `null`
All matched strings	`str.match(regexWithG)`	array of strings or **`null`**
All matches with details	`str.matchAll(regexWithG)` (ES2020)	iterator of match arrays (throws without `g`)
Position of first match	`str.search(regex)`	index or -1
Replace	`str.replace(regex, replacement)` / `replaceAll`	new string
Split	`str.split(regex)`	array

`test`, `match`, `matchAll` and named groups

From `code/04_regex_basics.mjs`:

```
const literal = /colou?r/i;
const dynamic = new RegExp('colou?r', 'i');
console.log(literal.test('Nice COLOR'), dynamic.source, dynamic.flags, literal.toString());

const zip = /^\d{5}(-\d{4})?$/;
console.log(['12345', '12345-6789', '1234', '12345-'].map((z) => `${z}:${zip.test(z)}`).join(' '));

const text = 'Order #123 shipped, order #456 pending';
const first = text.match(/#(\d+)/);
console.log(first[0], first[1], first.index);
console.log(text.match(/#\d+/g), 'no match ->', text.match(/#x/g));

for (const m of text.matchAll(/#(?<id>\d+) (?<status>\w+)/g)) {
  console.log(m.index, m.groups.id, m.groups.status);
}

const { groups: { year, month, day } } = '2024-07-04'.match(/(?<year>\d{4})-(?<month>\d{2})-(?<day>\d{2})/);
console.log(year, month, day);
```

Output:

```
true colou?r i /colou?r/i
12345:true 12345-6789:true 1234:false 12345-:false
#123 123 6
[ '#123', '#456' ] no match -> null
6 123 shipped
26 456 pending
2024 07 04
```

- `u?` makes `u` optional; `i` ignores case.
- The ZIP pattern is **anchored** `^…$` — otherwise `test` accepts `'abc12345xyz'` (a match anywhere counts). Validation needs anchors.
- `match` without `g`: full match, groups, position; with `g`: just strings. **No match returns** `null`, not `[]` — use `str.match(re) ?? []` or `matchAll`.
- **Named groups** self-document and pair with destructuring.

`replace` with patterns and functions

```
console.log('2024-07-04'.replace(/(\d{4})-(\d{2})-(\d{2})/, '$3/$2/$1'));
console.log('2024-07-04'.replace(/(?<y>\d{4})-(?<m>\d{2})-(?<d>\d{2})/, '$<d>.$<m>.$<y>'));
console.log('price: 10 USD, 25 USD'.replace(/(\d+) USD/g, (all, n) => `€${(n * 0.9).toFixed(2)}`));
console.log('snake_case_name'.replace(/_(\w)/g, (_, c) => c.toUpperCase()));
console.log('a.b.c'.replaceAll('.', '/'), 'x'.replace('x', '$&$&'));
console.log('one, two;three  four'.split(/[,;\s]+/), '1+2-3'.split(/([+-])/));
```

Output:

```
04/07/2024
04.07.2024
price: €9.00, €22.50
snakeCaseName
a/b/c xx
[ 'one', 'two', 'three', 'four' ] [ '1', '+', '2', '-', '3' ]
```

Replacement-**string** patterns: `$1`…`$99`, `$<name>`, `$&` (whole match), `` $` ``/`$'` (text before/after), `$$` (literal `$`) — applied even for plain-string search patterns, so pass user text through a **function** instead. A replacement function receives `(match, group1, group2, …, offset, fullString, groups)` and returns the insert — the most powerful form. Splitting by a regex with a **capturing group** keeps the captured separators.

Greedy vs lazy, anchors, flags in action

```
const html = '<b>bold</b> and <i>italic</i>';
console.log(html.match(/<.+>/)[0], '|', html.match(/<.+?>/)[0], '|', html.match(/<[^>]+>/g));

console.log(/\bcat\b/.test('concatenate'), /\bcat\b/.test('a cat sat'));
const lines = 'id: 1\nname: Bo\nid: 2';
console.log(lines.match(/^id: \d+$/g), lines.match(/^id: \d+$/gm));

console.log(/start.+end/.test('start\nend'), /start.+end/s.test('start\nend'));
```

Output:

```
<b>bold</b> and <i>italic</i> | <b> | [ '<b>', '</b>', '<i>', '</i>' ]
false true
null [ 'id: 1', 'id: 2' ]
false true
```

- Quantifiers are **greedy**: `.+` grabs all it can then backs off, so `<.+>` runs to the *last* `>`; `?` makes it **lazy**. A negated class (`[^>]+`) is clearer and faster.
- `\b` keeps `cat` from matching inside `concatenate`.
- Without `m`, `^`/`$` mean the whole input; with `m`, each line.
- `.` skips `\n` without the `s` flag.

(Real HTML needs a **parser** — reads a format into a structure — such as `DOMParser`; regexes are fragile.)

Lookaround and backreferences

```
console.log('$30 and €40 and $5'.match(/(?<=\$)\d+/g));        // numbers preceded by $
console.log('100px 2em 30px'.match(/\d+(?=px)/g));             // numbers followed by px
const strong = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$/;
console.log(['weakpass', 'Str0ngPass', 'Short1A'].map((p) => `${p}:${strong.test(p)}`).join(' '));
console.log('this is is a test test'.match(/\b(\w+) \1\b/g));
```

Output:

```
[ '30', '5' ]
[ '100', '30' ]
weakpass:false Str0ngPass:true Short1A:false
[ 'is is', 'test test' ]
```

Lookarounds test what's before/after a position *without including it*. Stacked lookaheads (`(?=.*[a-z])(?=.*[A-Z])(?=.*\d)`) express "must contain each" — the classic password rule. Backreference `\1` = "same text as group 1" — finds doubled words.

The `lastIndex` trap and `exec` loops

```
const g = /a/g;
console.log(g.test('a'), g.lastIndex, g.test('a'), g.lastIndex, g.test('a'));

const re = /\d+/g; let m; const found = [];
while ((m = re.exec('a1b22c333')) !== null) found.push(`${m[0]}@${m.index}`);
console.log(found);

const sticky = /\d+/y; sticky.lastIndex = 1;
console.log(sticky.exec('a12b')?.[0], (sticky.lastIndex = 0, sticky.exec('a12b')));
```

Output:

```
true 1 false 0 true
[ '1@1', '22@3', '333@6' ]
12 null
```

A `g` (or `y`) regex stores where its last match ended in `lastIndex`, and `test`/`exec` resume from there — so reusing one global regex to validate several strings alternates `true`/`false`. Fixes: no `g` for `test`, reset `lastIndex = 0`, or `matchAll` (which copies the regex). The `while (exec)` loop is the pre-2020 way to walk matches; `matchAll` replaces it. **Sticky** `y` matches only exactly at `lastIndex` — a tokenizer building block.

6.9 Modern regex features and practical recipes

From `code/05_regex_modern.mjs`:

```
console.log(/^.$/.test('😀'), /^.$/u.test('😀'));
console.log('Ünïcödé 123 ñ'.match(/\p{L}+/gu));
console.log('Price: ৳ or 34'.match(/\p{Nd}+/gu));
console.log('Hi 👋 there 🎉'.replace(/\p{Extended_Pictographic}/gu, '*'));

console.log('abcdef'.match(/[\p{L}--[aeiou]]/gv));
console.log('I 🇮🇳 flags'.match(/\p{RGI_Emoji_Flag_Sequence}/v)?.[0]);

const userInput = 'a.b*c';
const safe = new RegExp(RegExp.escape(userInput), 'g');
console.log(RegExp.escape(userInput), 'xa.b*cx axbbc'.replace(safe, '[hit]'));
console.log('without escaping:', 'xa.b*cx axbbc'.replace(new RegExp(userInput, 'g'), '[hit]'));

const withIdx = /(?<key>\w+)=(?<val>\w+)/d.exec('  mode=dark');
console.log(withIdx.indices.groups.key, withIdx.indices.groups.val);

const dateRe = /(?<y>\d{4})-(?<m>\d{2})|(?<m>\d{2})\/(?<y>\d{4})/;
console.log('2024-07'.match(dateRe).groups, '07/2024'.match(dateRe).groups);

console.log(/^(?i:hello) World$/.test('HELLO World'), /^(?i:hello) World$/.test('HELLO WORLD'));
```

Output:

```
false true
[ 'Ünïcödé', 'ñ' ]
[ '৳', '34' ]
Hi * there *
[ 'b', 'c', 'd', 'f' ]
🇮🇳
\x61\.b\*c x[hit]x axbbc
without escaping: xa.b*cx [hit]
[ 2, 6 ] [ 7, 11 ]
[Object: null prototype] { y: '2024', m: '07' } [Object: null prototype] { y: '2024', m: '07' }
true false
```

Feature	Version	What it gives you
`u` flag + `\p{…}` property escapes	ES2015 / ES2018	Code-point matching; letters, digits, emoji in **any** script (`\w` and `\d` are ASCII-only)
`v` flag	ES2024	Class **set operations** — difference `--`, intersection `&&`, nested classes — and multi-code-point properties like `\p{RGI_Emoji_Flag_Sequence}`
`d` flag	ES2022	`indices` with `[start, end]` for the match and each group — for editors and highlighters
`RegExp.escape(str)`	**ES2025**	Escapes every regex-special character so user text is matched literally. Note it also hex-escapes a leading letter or digit (`\x61`) so the result is safe even when placed after another escape
Duplicate named groups	**ES2025**	The same group name may appear in different `
Pattern modifiers `(?i:…)`	**ES2025**	Turn `i`, `m` or `s` on/off for part of a pattern

All run in Node 24 and current evergreen (self-updating) browsers. Without `RegExp.escape`, the user's `.` and `*` acted as operators and matched the wrong text — an injection bug, and with patterns like `(a+)+` a potential **ReDoS** (regular-expression denial of service: exponential-time matching that freezes the thread).

Practical recipes

```
const logs = `127.0.0.1 - [10/Oct/2024:13:55:36] "GET /index.html HTTP/1.1" 200 2326
10.0.0.5 - [10/Oct/2024:13:56:01] "POST /api/login HTTP/1.1" 401 87`;
const lineRe = /^(?<ip>\S+) - \[(?<time>[^\]]+)\] "(?<method>[A-Z]+) (?<path>\S+) [^"]+" (?<status>\d{3}) (?
<bytes>\d+)$/gm;
console.table([...logs.matchAll(lineRe)].map((m) => ({ ...m.groups, status: Number(m.groups.status) })));

const email = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;
console.log(['a@b.co', 'no-at.com', 'x@y', 'sp ace@a.io'].map((e) => `${e}:${email.test(e)}`).join(' '));

const n = 1234567.891;
console.log(String(Math.trunc(n)).replace(/\B(?=(\d{3})+(?!\d))/g, ','), n.toLocaleString('en-US'),
n.toLocaleString('de-DE'));

const tpl = 'Hi {name}, your code is {code}. {unknown}';
const data = { name: 'Ana', code: 'X9' };
console.log(tpl.replace(/\{(\w+)\}/g, (m, key) => data[key] ?? m));
```

Output:

```
┌─────────┬─────────────┬──────────────────────┬──────────┬──────────────┬──────────┬──────────┐
│ (index) │ ip          │ time                 │ method   │ path         │ status   │ bytes    │
├─────────┼─────────────┼──────────────────────┼──────────┼──────────────┼──────────┼──────────┤
│ 0       │ '127.0.0.1' │ '10/Oct/2024:13:55:36' │ 'GET'  │ '/index.html' │ 200     │ '2326'   │
│ 1       │ '10.0.0.5'  │ '10/Oct/2024:13:56:01' │ 'POST' │ '/api/login'  │ 401     │ '87'     │
└─────────┴─────────────┴──────────────────────┴──────────┴──────────────┴──────────┴──────────┘
a@b.co:true no-at.com:false x@y:false sp ace@a.io:false
1,234,567 1,234,567.891 1.234.567,891
Hi Ana, your code is X9. {unknown}
```

1. **Log parsing**: `gm` + named groups + `matchAll` turn lines into objects — the idea behind log pipelines (Logstash "grok" patterns, Fluent Bit parsers).
2. **Email**: a *pragmatic* check ("something@something.tld", no spaces; the **TLD** — top-level domain — is the last part like `.com`). Full RFC 5322 (the email-format standard) validation by regex is impractical; the real test is a confirmation email. `<input type="email">` uses a similar simplified rule.
3. **Thousands separators**: lookahead `(?=(\d{3})+(?!\d))` finds positions before groups of three digits — neat, but `toLocaleString` / `Intl.NumberFormat` is correct for every locale.
4. **Templating**: a replacer function looks up each `{key}`, leaving unknown keys.

Trade-offs: regex or string methods?

Situation	Prefer
Fixed substring test / replace	`includes`, `startsWith`, `replaceAll` — clearer and faster
Patterns (digits, optional parts, alternatives)	Regex
Structured formats (HTML, JSON, CSV with quotes, URLs)	A real parser: `DOMParser`, `JSON.parse`, a CSV library, `new URL()`
Human-language text (words, characters)	`Intl.Segmenter`, `\p{…}` with `u` / `v`

When not to use it

Avoid giant unreadable regexes — split into steps or write a small parser. Never run a user-supplied pattern without a timeout (or a linear-time engine like RE2, whose time grows only in proportion to

input): **catastrophic backtracking** can hang a Node server — the cause of real outages such as Cloudflare's July 2019 outage.

Common mistakes

- No anchors in validation (`/\d{5}/.test('abc123456')` is `true`).
- Forgetting `g` in `replace`; `matchAll`/`replaceAll` with a non-global regex (both throw `TypeError`).
- Reusing a `/g` regex with `test` (`lastIndex`).
- `new RegExp(userText)` without `RegExp.escape`.
- `'\d'` instead of `'\\d'` in the constructor string.
- Expecting `\w` to match `é`/`ñ` — ASCII-only; use `\p{L}` with `u`.

Real-world examples

- **Routers**: Express (the most popular Node web framework) and front-end routers compile `/users/:id` into a regex with a capture group.
- **Linters/formatters** use regexes for simple rules; editors use `d` indices for highlighting.
- HTML `pattern` **attribute** validation is a regex (evaluated with `v` in modern browsers).
- **Search tools** (ripgrep, VS Code search) and log processing run on regex engines.

In the interview

30-second answer: "A regex is a pattern object — `/…/flags` or `new RegExp`. `test` for yes/no, `matchAll` with named groups to extract, `replace` with a callback to transform. Gotchas: anchors for validation, `g` for global replace, stateful `lastIndex` with `g`, greedy vs lazy, and `RegExp.escape` for user input."

- *`match` vs `matchAll`?* `match` with `g` returns only strings (or `null`); `matchAll` an iterator of full matches with groups and indexes.
- *Match a word in any language?* `/\p{L}+/u`.
- *Catastrophic backtracking?* Nested quantifiers like `(a+)+$` try exponentially many splits on near-matches; avoid nested unbounded quantifiers.

Self-check

1. `/^\d{3}-\d{4}$/` matches? → Three digits, dash, four digits (`555-1234`).
2. `'aaa'.replace(/a/, 'b')`? → `'baa'`.
3. Named group `year` in a replacement string? → `$<year>`.
4. Why is `/<.+>/` wrong for one tag? → Greedy; use `/<[^>]+>/`.
5. ES2025 method to escape user input? → `RegExp.escape`.

Chapter TL;DR

- Strings are **immutable UTF-16 primitives**: `'…'`/`"…"` (identical) or backticks (interpolation, multi-line, tags). Escapes: `\n`, `\t`, `\\`, `\'`, `\u{…}`.
- Index with `[i]`, `charAt`, `at(-1)`; out-of-range brackets → `undefined`. Index assignment never changes a string (throws in strict mode).
- `+` concatenates if either side is a string; other arithmetic converts to numbers. Avoid `new String`; primitives get methods via auto-boxing.
- Core methods: `includes`/`startsWith`/`endsWith`/`indexOf`, `slice`, `toUpperCase`/`toLowerCase`, `trim*`, `padStart`/`padEnd`, `repeat`, `split`/`join`, `replace` (first only)/`replaceAll`, `localeCompare`/`Intl.Collator`.
- `length` counts code units (emoji = 2). `for…of`/spread for code points; `Intl.Segmenter` for visible characters; `normalize('NFC')` before comparing; `isWellFormed`/`toWellFormed` for lone surrogates.

- Regexes: classes, quantifiers (greedy; `?` lazy), anchors, groups (numbered, named, non-capturing), lookaround, backreferences; flags `g i m s u v y d`.
- Methods: `test`, `match` (null on no match), `matchAll` (needs `g`), `replace` with `$1`/`$<name>` or a callback, `split` by regex. Beware `lastIndex` with `/g`.
- ES2024–2025: `v` flag, `RegExp.escape`, pattern modifiers, duplicate named groups — all in Node 24. String methods for fixed text, real parsers for structured formats.